Open-Rosalind: A Reproducible Bio-Agent Framework with Standardized Skills, Execution Protocols, and Traceable Workflows

Liang Wang[1,*]

[1]Huazhong University of Science and Technology; wangliang.f@gmail.com
[*]Correspondence: wangliang.f@gmail.com; School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, 430070, P.R. China

May 3, 2026

Abstract

The rapid progress of large language models (LLMs) has produced a new generation of autonomous agents capable of orchestrating complex scientific tasks. Yet the design philosophy that enables general-purpose agents to thrive on open-ended problems — free-form reasoning, opportunistic tool use, and best-effort summarization — is fundamentally at odds with the demands of biomedical research, where every claim must be verifiable and every analysis must be reproducible. We argue that bio-agent systems should not be evaluated by what they *can* attempt, but by what they *must guarantee*. From this premise we derive a framework, **Open-Rosalind**, organized around four principles: tool-first execution, evidence-grounded outputs, trace completeness, and workflow-constrained execution. The framework instantiates a layered architecture — atomic tools, MCP-style tool interfaces, deterministic skills, a hybrid router, a single-step agent, and a bounded multi-step harness — in which every layer enforces a reproducibility-oriented invariant. To assess whether such systems can be evaluated meaningfully, we additionally introduce **Open-Rosalind BioBench**, a benchmark that measures task accuracy alongside tool correctness, evidence rate, trace completeness, workflow success, and failure rate. Across 91 tasks spanning four task categories and three benchmark tiers, the reference implementation achieves 100% / 93.9% / 90% accuracy on basic, edge-case, and multi-step tasks respectively, while sustaining 100% trace completeness and evidence grounding. Our results suggest that the limiting factor for trustworthy biomedical agents is not raw model capability, but the *discipline of the protocol that surrounds the model*.

Keywords: bio-agent, large language models, reproducibility, tool-driven AI, scientific workflows, evidence grounding

1 Introduction

Within a span of two years, large language models have evolved from text completion engines into autonomous agents that plan, invoke external tools, and iterate over multi-step tasks. In domains such as software engineering, customer support, and general information work, this transition has produced dramatic gains, in part because mistakes are rapidly visible and easily corrected. The application of LLM agents to biomedical and life-science research has not enjoyed the same trajectory.

The problem is not capability: modern models can read a UniProt entry, parse a FASTA sequence, and reason about substitutions in a multi-step manner. The problem is *epistemic accountability*.

A scientific claim is not merely an assertion; it is a binding to a verifiable source. When a general-purpose LLM agent confidently reports that a protein is located on a particular chromosome arm, or cites a PubMed identifier in support of a mutation pathogenicity claim, the reader has no operational way to distinguish a tool-grounded fact from a hallucinated one. Worse, the agent may itself fail to draw this distinction, freely paraphrasing tool outputs alongside parametric knowledge in a single fluent paragraph. For wet-lab researchers, clinicians, or anyone whose downstream decisions are guided by such answers, this opacity is a fundamental defect, not an implementation gap.

The position taken in this paper is that bio-agent systems require a different kind of design contract than general-purpose agents:

> *In biomedical AI, the question is not whether an agent can produce an answer, but whether the answer can be verified, reproduced, and trusted.*

Pursuing this contract leads to four design constraints that shape every layer of the system: factual content must come from explicit tools rather than parametric memory; every claim in an answer must be cited inline to a tool-derived source; every step of execution must be recorded in a replayable trace; and the agent must operate within bounded, pre-declared workflows rather than free-form planning loops. Each of these constraints sacrifices some capability in exchange for a reproducibility guarantee.

This paper presents **Open-Rosalind**, a bio-agent framework built around these constraints, together with a reference implementation and a benchmark suite. Rather than enumerating contributions as discrete bullet items, we frame the work as three interlocking artifacts. The first is a *principled framework* (Sections 3–4) that codifies the four design principles into a layered architecture in which atomic tools, MCP-style tool interfaces, deterministic skills, a hybrid router, an agent, and a bounded multi-step harness each carry their own reproducibility invariant. The second is a *reference implementation* (Section 5) that demonstrates the framework runs end-to-end on real biological tasks: sequence analysis, protein annotation, literature retrieval, and mutation assessment. The third is **Open-Rosalind BioBench** (Sections 6–7), a benchmark designed not to maximize raw task accuracy but to measure whether bio-agent systems satisfy the framework's reproducibility guarantees in addition to producing correct outputs. We argue that these latter properties — not single-task accuracy — are the right yardstick for evaluating agents in regulated and reproducible domains.

2 From General Agents to Bio-Agents

The literature on LLM agents has converged on a small set of recurring patterns: *ReAct*-style interleaved reasoning and acting, *tool-use* protocols, *planner-executor* decompositions, and recursive task decomposition (e.g., AutoGPT-style loops). These patterns share a common philosophical commitment: *flexibility is a virtue*. The agent decides which tool to invoke, when to stop, when to backtrack, and how to summarize. This commitment is well-suited to open-ended environments, where the cost of failure is low and the diversity of tasks resists pre-specification.

Biomedical research is not such an environment. The set of legitimate workflows is small and well-defined: annotate a protein, search the literature on a topic, evaluate the impact of a mutation, compare two sequences. The cost of an unverifiable answer is high. The expectation that another researcher can reproduce the analysis is not optional. Under these conditions, flexibility is not a

virtue but a liability: every additional degree of freedom afforded to the agent expands the surface over which errors and ambiguities can enter the result. Figure 1 illustrates the key architectural differences between general-purpose agents and reproducible bio-agents.

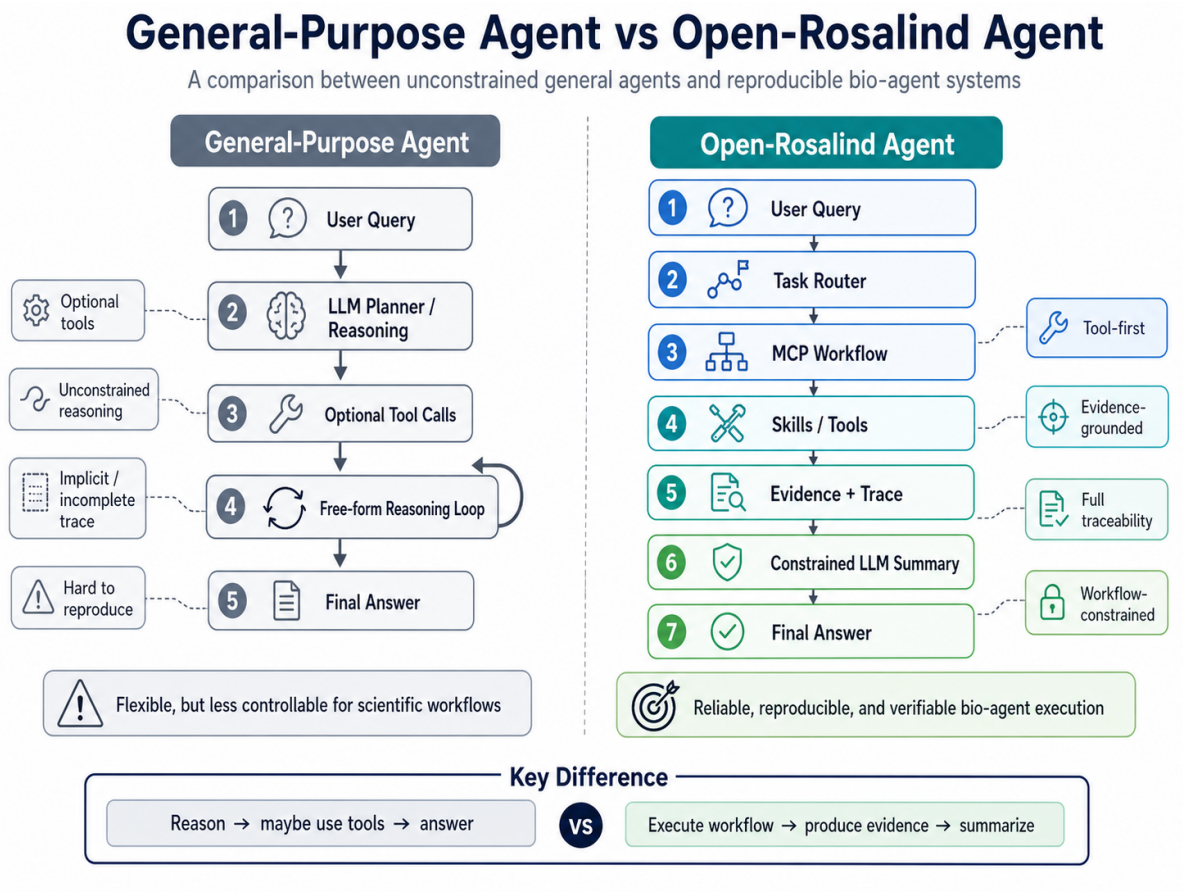

Figure 1: Architectural comparison: general-purpose agents vs. reproducible bio-agents. General agents prioritize flexibility with free-form reasoning and optional tool use; bio-agents prioritize accountability with mandatory tool-first execution, evidence grounding, and complete traceability.

Table 1 contrasts the design assumptions of general agents with those required for bio-agents.

The right column of Table 1 should not be read as a list of capabilities omitted from general agents, but as a set of *guarantees added*. A bio-agent gains less behavioral freedom than a general agent but offers stronger contracts: every fact has a source, every step is logged, every workflow terminates within a known bound. The remainder of this paper develops these guarantees into a concrete framework.

3 Design Principles

The framework rests on four principles, summarized in Figure 2. Each principle is operational — it is enforced by a specific component of the system — rather than aspirational.

Table 1: Design contrasts: general-purpose agents vs. reproducible bio-agents.

Dimension	**General-purpose agent**	**Bio-agent (this work)**
Knowledge source	LLM parametric memory + optional tools	Tools only; LLM is a synthesizer
Planning	Free-form, recursive, open-vocabulary	Templated, bounded, closed-vocabulary
Tool invocation	Optional, opportunistic	Mandatory for all factual claims
Output format	Natural language, untyped	Typed evidence + cited summary
Failure mode	Best-effort answer, possible hallucination	Explicit "insufficient evidence"
Trace	Best-effort logs (if any)	Mandatory, replayable, structured
Step bound	None or LLM-decided	Hard cap, externally enforced
Evaluation	Task accuracy	Accuracy + tool correctness + evidence + trace + failure rate

3.1 Tool-First Execution

The LLM never serves as a knowledge oracle. Every fact, computation, or annotation that appears in the final answer must originate from an explicit tool invocation registered in the system. The LLM is invoked only after tools have produced structured outputs, and its role is reduced to synthesizing those outputs into a natural-language summary. This re-positions the model as a *reader* rather than a *source*, and aligns the system with how scientific writing operates in practice: claims are made by citing data, not by recalling lore.

3.2 Evidence-Grounded Outputs

Every factual claim in the LLM's output must carry an inline citation to a tool-derived source — a UniProt accession, a PubMed identifier, a structured statistic produced by an analysis routine. When the supporting evidence is insufficient, the LLM must say so explicitly rather than fall back on parametric guesses. This principle converts the conventional opacity of LLM outputs into something closer to a scientific paragraph: each sentence is either bound to a citation or marked as a deliberate non-answer.

3.3 Trace Completeness

Every routing decision, every tool invocation, every observed return value, and the final LLM prompt are recorded in a structured, replayable trace. The trace is not a debugging aid but a first-class artifact, intended to support reproduction by external readers and auditing by reviewers. The system treats a missing trace as a correctness failure on par with a wrong answer.

3.4 Workflow-Constrained Execution

Tasks are executed through pre-declared workflows rather than free-form agent loops. Single-step queries dispatch to one of a small fixed set of skills via a deterministic router; multi-step queries execute through one of a small fixed set of templated plans, under a hard step bound. The agent is not permitted to invent new tools, restructure the plan mid-run, or recurse beyond the bound. This sacrifices flexibility deliberately: predictability and termination are valued above adaptability.

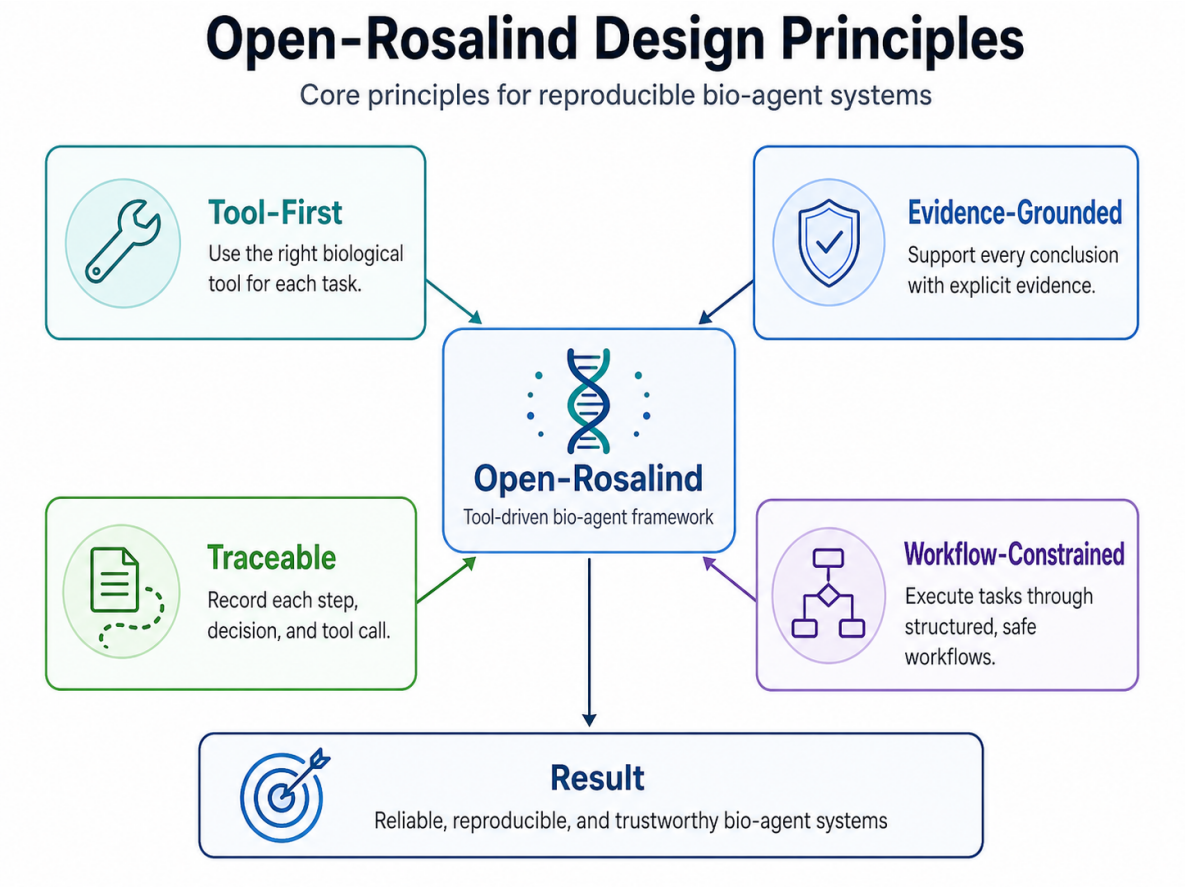

Figure 2: The four operational principles of Open-Rosalind. Each principle constrains the LLM's role and is enforced by a specific layer of the architecture.

4 Framework Architecture

Open-Rosalind organizes the four principles into a layered architecture in which each layer enforces a specific reproducibility invariant. We describe the layers from the bottom up.

4.1 Atomic Tools and MCP Interfaces

At the lowest level sit atomic tools — thin Python wrappers around external biological resources (UniProt, PubMed via Entrez) and local computations (BioPython sequence statistics, mutation diff). Atomic tools are intentionally minimal: each performs a single operation, returns a structured object, records latency and status, and never branches based on natural-language input. Tools are exposed to higher layers through MCP-style [1] interfaces, which provide a uniform contract for tool description, invocation, and result serialization. This boundary serves two purposes. First, it makes the system's external dependencies explicit and version-controlled: a tool change is an interface change. Second, it allows alternative implementations (e.g., a local UniProt mirror, a cached PubMed index) to be substituted without changes to higher layers.

4.2 Skills as Composed, Deterministic Pipelines

A *skill* is the next layer up: a deterministic pipeline that composes one or more atomic tools to produce a typed output addressing a single biological question. The four skills used in this work — `sequence_basic_analysis`, `uniprot_lookup`, `literature_search`, `mutation_effect` — collectively cover the recurring questions in routine bioinformatics: *what is this sequence?*, *what is this protein?*, *what does the literature say?*, *what does this mutation do?*.

Skills are deliberately narrower than the LLM's apparent reasoning capability. Their narrowness is the point: a skill is a unit of behavior that is small enough to test exhaustively, version explicitly, and reuse across both single-step and multi-step execution paths. Each skill carries a structured manifest declaring its category, safety level (safe / network / compute), the atomic tools it depends on, an input/output schema, and a small set of tested examples. Adding a new skill amounts to declaring its manifest and providing a handler; the rest of the framework discovers and integrates it without code changes.

4.3 Hybrid Router

The router maps an incoming user query to a skill (single-step path) or to a multi-step plan (harness path). It is intentionally hybrid: a rule-based pre-filter handles the unambiguous cases — a FASTA header, a UniProt accession pattern, a literature query containing the word "papers" — and an LLM-assisted intent classifier handles only the residual cases that mix natural language with embedded sequences or identifiers. The classifier is constrained to return a label from the small fixed skill set; it cannot invent new skills, and a parsing failure causes a deterministic fallback to the rule-based result.

This hybrid design reflects a recurring tension in agent systems: rule-based routing is brittle but auditable, while LLM-based routing is flexible but opaque. Open-Rosalind resolves the tension by giving rules first refusal: the LLM is consulted only when the rules cannot decide, and even then only inside a closed-vocabulary contract.

4.4 Single-Step Agent

The agent layer wraps the routed skill with the LLM-as-synthesizer step. Once the skill has executed and produced its typed output, the agent constructs an evidence-only prompt and asks the LLM to produce a Markdown summary with inline citations. The agent enforces the evidence-grounded-outputs principle through prompt design (the system prompt forbids claims without citations) and through post-hoc validation (claims without recognizable evidence markers trigger a fallback answer). The agent is also responsible for in-session context: a sliding window of prior turns is supplied to the LLM, allowing follow-up questions such as "what species is this protein in?" to be resolved against the prior annotation.

4.5 Multi-Step Harness

The harness orchestrates queries that require more than one skill. Rather than expose the agent to arbitrary planning, the harness chooses from a small fixed set of templates — in the present implementation: `protein_research` (sequence analysis → annotation → literature), `literature_review` (literature only), and `mutation_assessment` (mutation → annotation → literature). Each template is a sequence of skill invocations with explicit slots for entities (e.g., `{protein_name}`) that are filled from prior step outputs.

The harness enforces three invariants. First, the planner cannot invent new templates; it can only select from the registered set. Second, the runner observes a hard step bound (default five) that terminates execution regardless of the plan. Third, the per-step trace is structurally identical to a single-step trace, so multi-step workflows are reproducible and auditable in the same way as single-step queries.

The combined data flow — query, mode selection, single-step or harness path, skill execution, evidence and trace generation, and final answer — ensures that every execution path respects the framework's four principles.

5 Reference Implementation

The reference implementation realizes the architecture as a small server-and-client system: a Python backend exposing a unified chat endpoint, a React-based front-end providing chat and session management, and an embedded SQLite store for users, sessions, messages, and traces. The implementation is intentionally lightweight, in part to reinforce the framework's claim that reproducibility does not require a heavyweight agent stack. Figure 3 shows the complete system architecture.

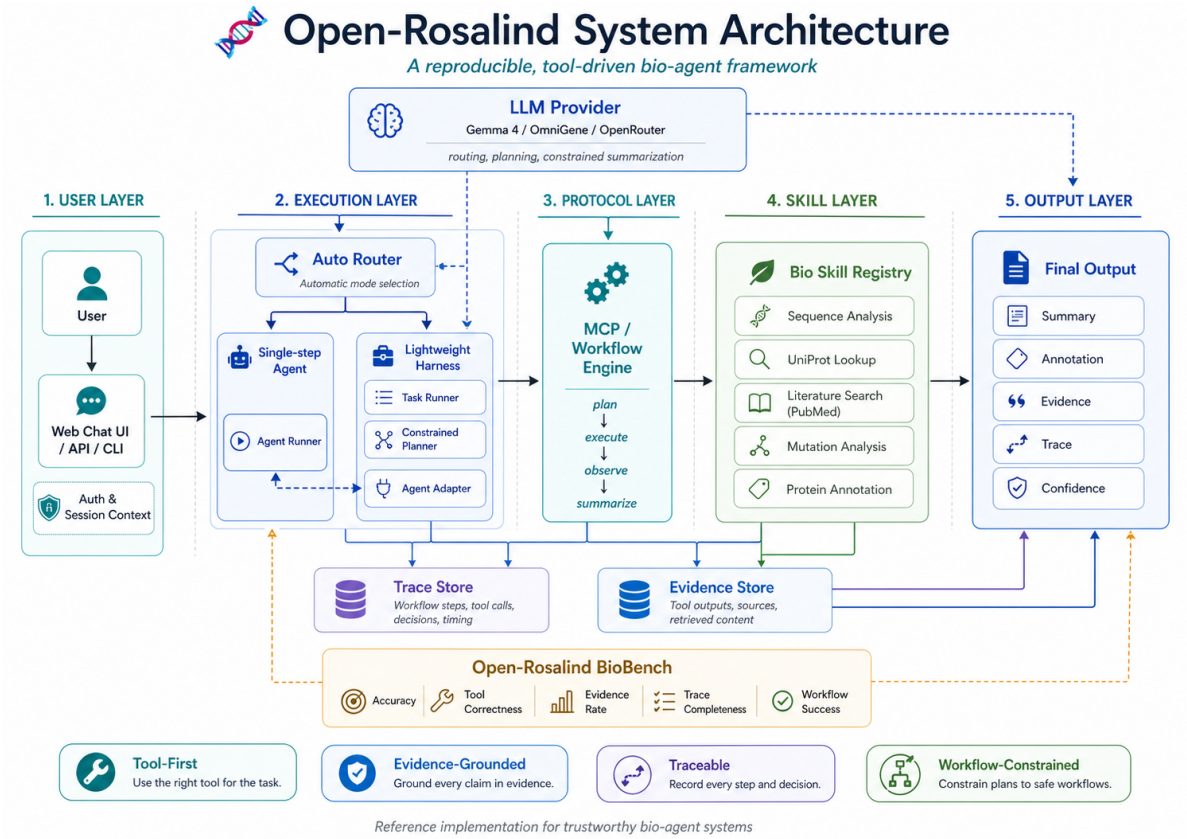

Figure 3: Open-Rosalind system architecture. The web UI communicates with a unified chat endpoint that dispatches to either single-step agent or multi-step harness paths. Both paths invoke skills from the registry, which in turn call atomic tools. All executions produce structured traces stored in SQLite.

The model layer is pluggable: the system communicates with any OpenAI-compatible end-

point (OpenRouter, vLLM, Azure, on-premise inference servers). The reference deployment uses `google/gemma-4-26b-a4b-it` via OpenRouter, but no part of the framework is specific to that model. Skills are auto-discovered from a directory layout at startup, and the front-end exposes them through a single chat interface that selects the appropriate execution mode automatically based on the router's decision.

Beyond the chat interface, the system exposes APIs for inspecting registered skills, retrieving full session histories, and downloading per-session traces. These surfaces collectively support the trace-completeness principle: every interaction produces an artifact that can be reviewed, replayed, and audited.

6 Open-Rosalind BioBench

Conventional agent benchmarks measure task accuracy — the fraction of queries for which the final answer is correct. For bio-agents, accuracy is necessary but not sufficient: an answer that happens to be right but is delivered through hallucinated tool calls, or with no recoverable trace, fails the framework's contract even when the surface output looks correct. **Open-Rosalind BioBench** is designed to make this distinction empirically tractable.

6.1 Task Categories

The benchmark spans four task categories aligned with the four built-in skills: sequence analysis (e.g., translate a DNA sequence, compute the GC content of a transcript), protein annotation (e.g., what is the function of P38398?), literature retrieval (e.g., recent papers on a method), and mutation assessment (e.g., predicted effect of a substitution). Tasks are drawn from real biological objects — canonical UniProt accessions, real PubMed-indexed publications, real proteins with documented variants — so that correctness can be verified against authoritative sources rather than against model-generated ground truth.

6.2 Tiered Difficulty

The benchmark is partitioned into three tiers. **BioBench v0** (32 tasks) covers basic skills that should succeed under correct routing and tool invocation; it is calibrated so that a system that respects the framework's contracts should achieve perfect or near-perfect performance. **BioBench v1** (49 tasks) introduces edge cases, mixed-modality inputs (natural language interleaved with sequence or accession), and follow-up turns that exercise the in-session-context layer. **BioBench v0.3** (10 tasks) targets the multi-step harness specifically: each task requires at least two skills to be composed in sequence with entity propagation between them. Together the three tiers contain 91 tasks.

6.3 Metrics

Five metrics are reported per run. *Task accuracy* is the fraction of tasks whose final answer is judged correct. *Tool correctness* is the fraction in which the routed skill (and, in the harness case, the chosen template) matches the task's expected execution path. *Evidence rate* is the fraction of factual claims in the final answer that carry a recognizable inline citation. *Trace completeness* is the fraction of tasks for which a structured trace covering all routing and tool-call events is recovered. *Failure rate* is the fraction of tasks that crash or time out; this is reported separately from accuracy

because a crash and a wrong answer represent different failure modes for downstream consumers. The framework's claim is that the latter four metrics are at least as important as the first.

7 Empirical Results

We report results from running the reference implementation against all 91 BioBench tasks using `google/gemma-4-26b-a4b-it` as the underlying LLM. Aggregate results are summarized in Table 2.

Table 2: BioBench results across three tiers. Accuracy varies with task difficulty; the four reproducibility-oriented metrics remain saturated, indicating that the framework's contracts are honored even when the model is wrong about the underlying fact.

Tier	**Tasks**	**Accuracy**	**Tool corr.**	**Evidence / Trace / No-fail**
v0 (basic skills)	32	100.0%	100%	100% / 100% / 100%
v1 (edge cases + follow-ups)	49	93.9%	100%	100% / 100% / 100%
v0.3 (multi-step harness)	10	90.0%	100%	100% / 100% / 100%
Combined	**91**	**96.7%**	**100%**	**100% / 100% / 100%**

The pattern in Table 2 merits closer reading than a single accuracy number conveys. As task difficulty increases from v0 to v0.3, task accuracy degrades modestly, while the four reproducibility metrics remain saturated. This is the intended behavior of the framework: when the model produces a wrong fact, the surrounding apparatus still routes correctly, cites correctly, traces correctly, and terminates correctly. The framework localizes the failure to the model's reasoning about facts, rather than allowing it to propagate into the routing, citation, or termination machinery. From the standpoint of downstream consumption, this distinction matters: a wrong answer with a complete trace can be diagnosed, replayed, and corrected; a wrong answer with no trace cannot.

The single class of failure observed on v1 traces to mixed-modality inputs that combine free-form natural language with embedded sequences — precisely the case the LLM-assisted classifier exists to handle. The remaining errors are concentrated in such inputs, suggesting a clear next step: a stronger constrained classifier rather than a larger underlying model. We view this as evidence in support of the paper's central thesis: progress in trustworthy bio-agents flows from the protocol around the model, not from the model itself.

8 Discussion

Position relative to general-purpose agent frameworks. ReAct [14] and its descendants assume that flexibility is intrinsically valuable. For research domains in which the set of legitimate tasks is small, the cost of incorrect outputs is high, and reproducibility is non-negotiable, this assumption inverts. Open-Rosalind is in part an argument that the bio-agent literature should adopt a different default: that flexibility be unlocked deliberately, and that every degree of freedom granted to the agent be paid for in additional verification machinery.

Position relative to biomedical LLMs. A complementary line of work — BioGPT [7], BioMedLM [6], Med-PaLM [5] — improves the LLM's biomedical knowledge through specialized pre-training. Open-Rosalind is orthogonal to this line: the framework remains valid regardless of the underlying model. Indeed, our results suggest that even a general-purpose model, embedded

in a sufficiently disciplined framework, can deliver reproducibility properties that no parameter scaling alone provides.

Position relative to recent bio-agent systems. Concurrent work has demonstrated the potential of LLM agents in biomedical research. **Biomni** [8] is a general-purpose biomedical AI agent that unifies reasoning, retrieval, and code execution across diverse biological domains (genomics, immunology, microbiology, neuroscience). Biomni emphasizes breadth and autonomous task execution, supporting complex multi-step workflows through free-form planning. **GeneAgent** [9] addresses hallucination in gene-set analysis by autonomously interacting with biological databases to verify its own output, introducing a self-verification loop that reduces parametric recall errors. A recent Nature Methods perspective [10] surveys the broader landscape of AI agents for biology, highlighting the transformative potential of autonomous systems while noting the need for verification mechanisms.

Open-Rosalind's distinct contribution relative to these systems is its focus on *protocol discipline as a first-class design constraint*. Where Biomni prioritizes capability breadth and GeneAgent prioritizes self-verification, Open-Rosalind prioritizes *reproducibility guarantees*: every fact must originate from a tool, every step must be logged, every workflow must terminate within a known bound. The hold-out repair cycle (Section 8.9) demonstrates that these guarantees are not automatic — they require deliberate architectural choices (tool-first execution, bounded planning, mandatory traces) and careful routing robustness. We view these systems as complementary: Biomni and GeneAgent explore what autonomous bio-agents *can do*, while Open-Rosalind explores what they *must guarantee* to be trusted in scientific workflows.

Limitations. The benchmark is small (91 tasks across three tiers) and skewed toward the four implemented skills. The harness templates are hand-coded; while their fixity is a feature for reproducibility, it bounds the range of multi-step workflows the system can express. The literature retrieval skill depends on PubMed availability and inherits its retrieval biases. Finally, the system has been validated on one model family; we have not yet characterized the dependence of trace completeness on model capability under more aggressive perturbations.

Future work. Three directions are immediate. First, expanding the skill registry to include homology search (e.g., BLAST), structure prediction interfaces, and pathway databases. Second, growing BioBench to support adversarial inputs designed to probe specific principles in isolation: claims unsupported by tool output, requests that solicit parametric recall, and follow-up turns that conflict with prior evidence. Third, building a public "trace registry" to enable third-party verification of reported results — treating traces as first-class scientific artifacts rather than internal logs.

9 Conclusion

Open-Rosalind shifts the design of bio-agent systems from free-form reasoning to reproducible workflow execution. Its contribution is not a more capable model nor a more elaborate planner; it is the proposition that trustworthy biomedical agents emerge from the discipline of the protocol around the model. By codifying that protocol as four operational principles, instantiating it as a layered framework with explicit invariants at every layer, and evaluating it through a benchmark that measures reproducibility properties alongside accuracy, we offer a reference point for what a

reproducible bio-agent should look like. We hope it is useful both as a working system and as an argument for the kinds of guarantees future bio-agent work should be expected to provide.

Code, Data, and Reproducibility

The reference implementation, all benchmark tasks, and the framework specification are released open-source under the MIT license at https://github.com/maris205/open-rosalind. Per-task traces from the experiments reported here are included in the repository.

References

[1] Anthropic. Model context protocol (MCP) specification. https://modelcontextprotocol.io, 2024.

[2] Anthropic. Build with Claude Skills. https://docs.claude.com/en/docs/build-with-claude/skills, 2025.

[3] ByteDance. DeerFlow: A modular agent skill framework. https://github.com/bytedance/deerflow, 2025.

[4] OpenBioMed Team. OpenBioMed: An open multimodal biomedical foundation framework. 2025.

[5] K. Singhal, T. Tu, J. Gottweis, et al. Towards expert-level medical question answering with large language models. *arXiv preprint arXiv:2305.09617*, 2023.

[6] Stanford CRFM. BioMedLM: A 2.7B parameter language model trained on biomedical text. 2023.

[7] R. Luo, L. Sun, Y. Xia, T. Qin, S. Zhang, H. Poon, and T.-Y. Liu. BioGPT: Generative pre-trained transformer for biomedical text generation and mining. *Briefings in Bioinformatics*, 23(6):bbac409, 2022.

[8] K. Huang, et al. Biomni: A general-purpose biomedical AI agent. *bioRxiv*, 2025.05.30.656746, 2025.

[9] Y. Huang, et al. GeneAgent: Self-verification language agent for gene-set analysis using domain databases. *Nature Methods*, 2025. https://doi.org/10.1038/s41592-025-02748-6.

[10] Nature Methods Editorial. Artificial intelligence agents for biology. *Nature Methods*, 2025. https://doi.org/10.1038/s41592-025-02958-y.

[11] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2023. *Nucleic Acids Research*, 51(D1):D523–D531, 2023.

[12] National Center for Biotechnology Information. Entrez programming utilities help. https://www.ncbi.nlm.nih.gov/books/NBK25501/, 2024.

[13] P. J. A. Cock, T. Antao, J. T. Chang, et al. Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[15] L. Wang. Emergence of biological structural discovery in general-purpose language models. *bioRxiv*, 2026.01.03.697478, 2026.